

From Research Code to Running Systems

Cross-Platform Robotics and ML Workflows

Tobias Fischer
Peter Corke

From Research Code to Running Systems

Cross-platform robotics and ML workflows

From Research Code to Running Systems

Cross-platform robotics and ML workflows

■ Research software is part of the research contribution.

From Research Code to Running Systems

Cross-platform robotics and ML workflows

■ Research software is part of the research contribution.

Today: turn fragile setup instructions into executable systems.

Robotics is Powered by Software

- Software controls the pace of robotics innovation
- Modern robotics requires integrating many ecosystems
- The bottleneck is often *not algorithms*, but environments

Robotics is Powered by Software

- Software controls the pace of robotics innovation
- Modern robotics requires integrating many ecosystems
- The bottleneck is often *not algorithms*, but environments

 What changes in practice?

Tooling becomes part of how research is shared, reused, and extended.

Robotics is Powered by Software

- Software controls the pace of robotics innovation
- Modern robotics requires integrating many ecosystems
- The bottleneck is often *not algorithms*, but environments

What changes in practice?

Tooling becomes part of how research is shared, reused, and extended.

Goal today

Move from:

- messy research repositories
- fragile installation instructions
- machine-specific setups

to:

- executable workflows
- cross-platform environments
- reproducible robotics systems others can extend

The Reality of Many Research Repositories

```
git clone https://github.com/some_repo/project

# Ubuntu 20.04 only
sudo apt install ...

pip install torch==1.12
pip install some_package

# Compile custom OpenCV

# Download model manually:
https://drive.google.com/...

# Tested on my machine :)
```


The Reality of Many Research Repositories

```
git clone https://github.com/some_repo/project

# Ubuntu 20.04 only
sudo apt install ...

pip install torch==1.12
pip install some_package

# Compile custom OpenCV

# Download model manually:
https://drive.google.com/...

# Tested on my machine :)
```

■ We can do better!

This Talk Is Executable

The slides are also the demo environment.

This Talk Is Executable

The slides are also the demo environment.

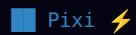
- Navigate with arrow keys
- Run code on a slide with `control` + `e`
- Show keybindings with `?`
- Exit with `esc` or `control` + `c`

This Talk Is Executable

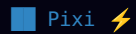
The slides are also the demo environment.

- Navigate with arrow keys
- Run code on a slide with `control` + `e`
- Show keybindings with `?`
- Exit with `esc` or `control` + `c`

Only run executable snippets from presentations you trust!



Pixi is a fast project manager built on the conda-forge ecosystem.



Pixi is a fast project manager built on the conda-forge ecosystem.

Think: conda packages, modern project workflow.



Pixi is a fast project manager built on the conda-forge ecosystem.

Think: conda packages, modern project workflow.

It provides:

- declarative environments
- lockfiles
- executable tasks
- cross-platform workflows

From Instructions to Artifacts

Instructions

- README.md
- shell scripts
- oral tradition
- "ask the previous student"

Artifacts

- `pixi.toml`
- `pixi.lock`
- `pixi run <task>`
- CI-friendly commands

From Instructions to Artifacts

Instructions

- README.md
- shell scripts
- oral tradition
- "ask the previous student"

Artifacts

- `pixi.toml`
- `pixi.lock`
- `pixi run <task>`
- CI-friendly commands

The shift is from instructions to
runnable artifacts.

■ Start a Project

```
pixi init
```

————— [finished] —————

```
✓ Created  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi/pixi.toml
```

■ Start a Project

```
pixi init
```

————— [finished] —————

```
✓ Created  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi/pixi.toml
```

■ Add PyTorch

```
pixi add pytorch torchvision
```

————— [finished] —————

```
✓ Added pytorch >=2.10.0,<3  
✓ Added torchvision >=0.26.0,<0.27
```

Fast Environments Change Behavior ⚡

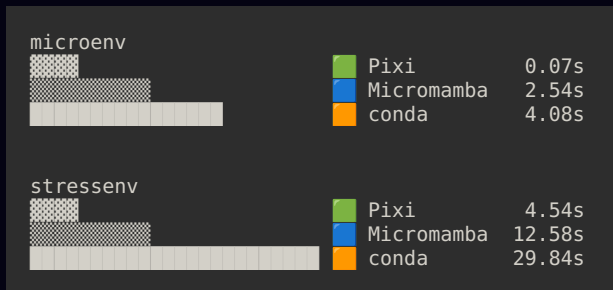
If changing environments takes `minutes`, people avoid changing environments.

Fast Environments Change Behavior ⚡

If changing environments takes **minutes**, people avoid changing environments.

If it takes **seconds**, environments become part of iteration.

Environment Solve Time

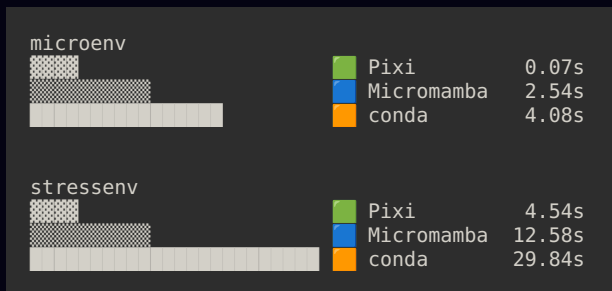


Fast Environments Change Behavior ⚡

If changing environments takes **minutes**, people avoid changing environments.

If it takes **seconds**, environments become part of iteration.

Environment Solve Time



Speed matters because research is exploratory.

```
import torch
import torch.nn.functional as F

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("MPS available:", torch.backends.mps.is_available())

y = torch.tensor([1.0])          # target
x = torch.tensor([1.1])          # input
w = torch.tensor([2.2], requires_grad=True)
b = torch.tensor([0.0], requires_grad=True)

z = x * w + b
a = torch.sigmoid(z)
loss = F.binary_cross_entropy(a, y)
loss.backward()

print("loss:", loss.item())
```

[finished]

```
PyTorch version: 2.10.0
CUDA available: False
MPS available: False
loss: 0.0851878821849823
```

Tasks Turn Commands into Workflows

 Define the workflow

```
pixi task add start "python train.py"  
--depends-on download-mnist
```

————— [finished] —————

```
✓ Added task `start`: python train.py,  
depends-on = 'download-mnist'
```


Tasks Turn Commands into Workflows

Define the workflow

```
pixi task add start "python train.py"  
--depends-on download-mnist
```

[finished]

✓ Added task `start`: python train.py,
depends-on = 'download-mnist'

```
[tasks]  
start = { cmd = "python train.py",  
depends-on = ["download-mnist"] }  
  
download-mnist = {  
  cmd = "python -c 'from  
torchvision.datasets import MNIST;  
MNIST(\"data\", download=True)'\",  
  outputs = ["data/MNIST"]  
}
```

Tasks Turn Commands into Workflows

Define the workflow

```
pixi task add start "python train.py"
--depends-on download-mnist
```

[finished]

✓ Added task `start`: python train.py,
depends-on = 'download-mnist'

```
[tasks]
start = { cmd = "python train.py",
depends-on = ["download-mnist"] }

download-mnist = {
  cmd = "python -c 'from
torchvision.datasets import MNIST;
MNIST(\"data\", download=True)'\",
  outputs = ["data/MNIST"]
}
```

Run it

```
pixi run start
```

[finished]

```
✨ Pixi task (download-mnist): python
-c 'from torchvision.datasets import
MNIST; MNIST("data", download=True)'
0.3%0.7%1.0%1.3%1.7%2.0%2.3%2.6%3.0%3.
3%3.6%4.0%4.3%4.6%5.0%5.3%5.6%6.0%6.3%
6.6%6.9%7.3%7.6%7.9%8.3%8.6%8.9%9.3%9.
6%9.9%10.2%10.6%10.9%11.2%11.6%11.9%12
.2%12.6%12.9%13.2%13.6%13.9%14.2%14.5%
14.9%15.2%15.5%15.9%16.2%16.5%16.9%17.
2%17.5%17.9%18.2%18.5%18.8%19.2%19.5%1
9.8%20.2%20.5%20.8%21.2%21.5%21.8%22.1
%22.5%22.8%23.1%23.5%23.8%24.1%24.5%24
.8%25.1%25.5%25.8%26.1%26.4%26.8%27.1%
27.4%27.8%28.1%28.4%28.8%29.1%29.4%29.
8%30.1%30.4%30.7%31.1%31.4%31.7%32.1%3
2.4%32.7%33.1%33.4%33.7%34.0%34.4%34.7
%35.0%35.4%35.7%36.0%36.4%36.7%37.0%37
.4%37.7%38.0%38.3%38.7%39.0%39.3%39.7%
40.0%40.3%40.7%41.0%41.3%41.7%42.0%42.
3%42.6%43.0%43.3%43.6%44.0%44.3%44.6%4
5.0%45.3%45.6%45.9%46.3%46.6%46.9%47.3
%48.6%48.9%49.3%49.6%49
```

Tasks Turn Commands into Workflows

Define the workflow

```
pixi task add start "python train.py"
--depends-on download-mnist
```

[finished]

✓ Added task `start`: python train.py,
depends-on = 'download-mnist'

```
[tasks]
start = { cmd = "python train.py",
depends-on = ["download-mnist"] }

download-mnist = {
  cmd = "python -c 'from
torchvision.datasets import MNIST;
MNIST(\"data\", download=True)'\",
  outputs = ["data/MNIST"]
}
```

Run it

```
pixi run start
```

[finished]

```
✨ Pixi task (download-mnist): python
-c 'from torchvision.datasets import
MNIST; MNIST("data", download=True)'\
0.3%0.7%1.0%1.3%1.7%2.0%2.3%2.6%3.0%3.
3%3.6%4.0%4.3%4.6%5.0%5.3%5.6%6.0%6.3%
6.6%6.9%7.3%7.6%7.9%8.3%8.6%8.9%9.3%9.
6%9.9%10.2%10.6%10.9%11.2%11.6%11.9%12
.2%12.6%12.9%13.2%13.6%13.9%14.2%14.5%
14.9%15.2%15.5%15.9%16.2%16.5%16.9%17.
2%17.5%17.9%18.2%18.5%18.8%19.2%19.5%1
9.8%20.2%20.5%20.8%21.2%21.5%21.8%22.1
%22.5%22.8%23.1%23.5%23.8%24.1%24.5%24
.8%25.1%25.5%25.8%26.1%26.4%26.8%27.1%
27.4%27.8%28.1%28.4%28.8%29.1%29.4%29.
8%30.1%30.4%30.7%31.1%31.4%31.7%32.1%3
2.4%32.7%33.1%33.4%33.7%34.0%34.4%34.7
%35.0%35.4%35.7%36.0%36.4%36.7%37.0%37
.4%37.7%38.0%38.3%38.7%39.0%39.3%39.7%
40.0%40.3%40.7%41.0%41.3%41.7%42.0%42.
3%42.6%43.0%43.3%43.6%44.0%44.3%44.6%4
5.0%45.3%45.6%45.9%46.3%46.6%46.9%47.3
%48.6%48.9%49.3%49.6%49
```

Mixing Ecosystems

Modern robotics often needs:

Robotics stack

- ROS
- OpenCV
- CUDA
- native libraries

■ Mixing Ecosystems

Modern robotics often needs:

▣ Robotics stack

- ROS
- OpenCV
- CUDA
- native libraries

▣ ML stack

- PyTorch
- Transformers
- experiment tools
- Python packages

■ Mixing Ecosystems

Modern robotics often needs:

▤ Robotics stack

- ROS
- OpenCV
- CUDA
- native libraries

▤ ML stack

- PyTorch
- Transformers
- experiment tools
- Python packages

■ Conda-forge + PyPI Together

```
[dependencies]
python = "3.11"
pytorch = "*"

[pypi-dependencies]
transformers = "*"
```

■ Mixing Ecosystems

Modern robotics often needs:

▤ Robotics stack

- ROS
- OpenCV
- CUDA
- native libraries

▤ ML stack

- PyTorch
- Transformers
- experiment tools
- Python packages

■ Conda-forge + PyPI Together

```
[dependencies]
python = "3.11"
pytorch = "*"

[pypi-dependencies]
transformers = "*"
```

■ No boundary between ecosystems ✨

Robotics Meets ML: RoboStack

RoboStack enables ROS on:

- Linux
- macOS
- Windows

through the conda-forge ecosystem.

Robotics Meets ML: RoboStack

RoboStack enables ROS on:

- Linux
- macOS
- Windows

through the conda-forge ecosystem.

```
pixi workspace channel add https://prefix.dev/robostack-rolling
pixi add ros-rolling-desktop
```

————— [finished] —————

```
✓ Added https://prefix.dev/robostack-rolling
WARN Skipped running the post-link scripts because
`run-post-link-scripts` = `false`
- bin/.librsvg-pre-unlink.sh
```

To enable them, run:

```
pixi config set --local run-post-link-scripts insecure
```

More info:

https://pixi.sh/latest/reference/pixi_configuration/#run-post-link-scripts

```
✓ Added ros-rolling-desktop >=0.13.0,<0.14
```

R0S Desktop App from a Locked Environment

Start with a tiny demo. Then scale the same idea to real stacks.

R0S Desktop App from a Locked Environment

Start with a tiny demo. Then scale the same idea to real stacks.

```
pixi run ros2 run turtlesim turtlesim_node
```

R0S Desktop App from a Locked Environment

Start with a tiny demo. Then scale the same idea to real stacks.

```
pixi run ros2 run turtlesim turtlesim_node
```

```
pixi run ros2 topic pub /turtle1/cmd_vel \  
  geometry_msgs/msg/Twist \  
  "{linear: {x: 2.0}, angular: {z: 1.8}}"
```

ROS + PyTorch + OpenCV
+ Transformers + custom research code

■ One Environment

```
ROS + PyTorch + OpenCV  
+ Transformers + custom research code
```

■ One lockfile.

■ One Environment

ROS + PyTorch + OpenCV
+ Transformers + custom research code

■ One lockfile.

■ One Machine, Multiple ROS Distro

```
# Add another ROS distro
pixi workspace channel add https://prefix.dev/robostack-humble
pixi add --feature humble ros-humble-desktop

# Run different environments
pixi run -e humble    ros2 run turtlesim turtlesim_node
pixi run -e rolling   ros2 run rviz2 rviz2

# We still support ROS1 Noetic :)
```

■ One Environment

```
ROS + PyTorch + OpenCV  
+ Transformers + custom research code
```

■ One lockfile.

■ One Machine, Multiple ROS Distro

```
# Add another ROS distro  
pixi workspace channel add https://prefix.dev/robostack-humble  
pixi add --feature humble ros-humble-desktop  
  
# Run different environments  
pixi run -e humble    ros2 run turtlesim turtlesim_node  
pixi run -e rolling   ros2 run rviz2 rviz2  
  
# We still support ROS1 Noetic :)
```

■ Different ROS distributions become environments, not separate machines.

From Environment to Infrastructure

This is where Pixi stops being only an environment manager.

From Environment to Infrastructure

This is where Pixi stops being only an environment manager.

It becomes lab infrastructure.

From Environment to Infrastructure

This is where Pixi stops being only an environment manager.

It becomes lab infrastructure.

Research code becomes something others can install.

Build Your Own ROS / C++ / Python Packages

Build Your Own ROS / C++ / Python Packages

■ Create a ROS package

```
ros2 pkg create \  
  --build-type ament_cmake \  
  --node-name icra_node \  
  icra_ros_package
```

————— [finished] —————

```
going to create a new package  
package name: icra_ros_package  
destination directory:  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['runner  
<runner@todo.todo>']  
licenses: ['TODO: License  
declaration']  
build type: ament_cmake  
dependencies: []  
node_name: icra_node  
creating folder ./icra_ros_package  
creating
```

Build Your Own ROS / C++ / Python Packages

 Create a ROS package

```
ros2 pkg create \  
  --build-type ament_cmake \  
  --node-name icra_node \  
  icra_ros_package
```

————— [finished] —————

```
going to create a new package  
package name: icra_ros_package  
destination directory:  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['runner  
<runner@todo.todo>']  
licenses: ['TODO: License  
declaration']  
build type: ament_cmake  
dependencies: []  
node_name: icra_node  
creating folder ./icra_ros_package  
creating
```

Build Your Own ROS / C++ / Python Packages

■ Create a ROS package

```
ros2 pkg create \  
  --build-type ament_cmake \  
  --node-name icra_node \  
  icra_ros_package
```

————— [finished] —————

```
going to create a new package  
package name: icra_ros_package  
destination directory:  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['runner  
<runner@todo.todo>']  
licenses: ['TODO: License  
declaration']  
build type: ament_cmake  
dependencies: []  
node_name: icra_node  
creating folder ./icra_ros_package  
creating
```

■ Build the package

```
pixi install
```

————— [finished] —————

```
└─ Running build for recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Fetching source code  
  No sources to fetch  
    (took 0  
seconds)  
└─ Externally resolved dependencies  
recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Resolved build  
dependencies(ros-rolling-icra-ros-pack  
age-0.0.0-hb0f4dca_0):
```

Build Your Own ROS / C++ / Python Packages

■ Create a ROS package

```
ros2 pkg create \  
  --build-type ament_cmake \  
  --node-name icra_node \  
  icra_ros_package
```

————— [finished] —————

```
going to create a new package  
package name: icra_ros_package  
destination directory:  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['runner  
<runner@todo.todo>']  
licenses: ['TODO: License  
declaration']  
build type: ament_cmake  
dependencies: []  
node_name: icra_node  
creating folder ./icra_ros_package  
creating
```

■ Build the package

```
pixi install
```

————— [finished] —————

```
└─ Running build for recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Fetching source code  
  No sources to fetch  
    (took 0  
seconds)  
└─ Externally resolved dependencies  
recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Resolved build  
dependencies(ros-rolling-icra-ros-pack  
age-0.0.0-hb0f4dca_0):
```


Build Your Own ROS / C++ / Python Packages

■ Create a ROS package

```
ros2 pkg create \  
  --build-type ament_cmake \  
  --node-name icra_node \  
  icra_ros_package
```

————— [finished] —————

```
going to create a new package  
package name: icra_ros_package  
destination directory:  
/home/runner/work/ICRA2026-modern-software-tools/ICRA2026-modern-software-tools/tobi  
package format: 3  
version: 0.0.0  
description: TODO: Package description  
maintainer: ['runner  
<runner@todo.todo>']  
licenses: ['TODO: License  
declaration']  
build type: ament_cmake  
dependencies: []  
node_name: icra_node  
creating folder ./icra_ros_package  
creating
```

■ Build the package

```
pixi install
```

————— [finished] —————

```
└─ Running build for recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Fetching source code  
  No sources to fetch  
    (took 0  
seconds)  
└─ Externally resolved dependencies  
recipe:  
ros-rolling-icra-ros-package-0.0.0-hb0  
f4dca_0  
└─ Resolved build  
dependencies(ros-rolling-icra-ros-pack  
age-0.0.0-hb0f4dca_0):
```

Cross Platform Reproducibility

- Same repository
- Same lockfile
- Same command
- Different machine
- Different OS

Cross Platform Reproducibility

- Same repository
- Same lockfile
- Same command
- Different machine
- Different OS

■ macOS → Linux → HPC

```
# Add other platforms  
pixi workspace platform add linux-64 win-64
```

Cross Platform Reproducibility

- Same repository
- Same lockfile
- Same command
- Different machine
- Different OS

■ macOS → Linux → HPC

```
# Add other platforms  
pixi workspace platform add linux-64 win-64
```

```
# Copy this demo to Linux/HPC via ssh/scp  
# Runs ssh/scp only when ICRA_REMOTE_HOST is set  
python helper.py remote-demo prepare
```

```
# Optional presenter remote: run the same Pixi task on Linux/HPC  
# Prints each remote command before executing it  
python helper.py remote-demo run
```

From Scripts to Infrastructure

```
graph TD; A[README.md + shell scripts] --> B[Executable workflows]; B --> C[Composable research systems];
```

README.md + shell scripts
↓
Executable workflows
↓
Composable research systems

From Scripts to Infrastructure

```
README.md + shell scripts
↓
Executable workflows
↓
Composable research systems
```

Better tooling changes how research is shared, reused, and extended.

Case Study: VSLAM-Lab

Large-scale visual SLAM framework for benchmarking, composability, reproducibility, and easy onboarding.

 Before 🙄

README.md

1. Install ROS
2. Build OpenCV
3. Download models
4. Download datasets
5. Configure CUDA
6. Build dependencies
7. Pray

 After ✨

```
git clone
https://github.com/VSLAM-LAB/VSLAM-LAB
.git > /dev/null 2>&1
cd VSLAM-LAB && \
pixi run demo orbslam2 eth table_3
mono
```


Case Study: VSLAM-Lab

Large-scale visual SLAM framework for benchmarking, composability, reproducibility, and easy onboarding.

 Before 🙄

README.md

1. Install ROS
2. Build OpenCV
3. Download models
4. Download datasets
5. Configure CUDA
6. Build dependencies
7. Pray

 After ✨

```
git clone
https://github.com/VSLAM-LAB/VSLAM-LAB
.git > /dev/null 2>&1
cd VSLAM-LAB && \
pixi run demo orbslam2 eth table_3
mono
```

Multi-page setup instructions become executable workflows.
Tooling stopped being a convenience.
It became research infrastructure.

Why Pixi for Robotics?

Built-in project feature	Pixi	Conda	Pip	Poetry	uv
Installs Python	✓	✓	✗	✗	✓
Native libraries	✓	✓	✗	✗	✗
Lockfiles	✓	via tools	✗	✓	✓
Task runner	✓	✗	✗	✗	✗
Workspace management	✓	✗	✗	✓	✓
Fast solver	✓	slower	n/a	✗	✓

Why Pixi for Robotics?

Built-in project feature	Pixi	Conda	Pip	Poetry	uv
Installs Python	✓	✓	✗	✗	✓
Native libraries	✓	✓	✗	✗	✗
Lockfiles	✓	via tools	✗	✓	✓
Task runner	✓	✗	✗	✗	✗
Workspace management	✓	✗	✗	✓	✓
Fast solver	✓	slower	n/a	✗	✓

The point is not one feature.

The point is having the right combination for robotics.

Why Not Docker?

 Containers are excellent for

- deployment
- cloud workflows
- CI/CD
- reproducible services
- production isolation

```
Dockerfile
apt-get
pip install
CUDA setup
X11 forwarding
volume mounts
```

Why Not Docker?

 Containers are excellent for

- deployment
- cloud workflows
- CI/CD
- reproducible services
- production isolation

```
Dockerfile
apt-get
pip install
CUDA setup
X11 forwarding
volume mounts
```

 But robotics research often needs

- native GUI applications
- low-friction hardware access
- ROS + ML composability
- rapid iteration across repositories
- cross-platform desktop workflows

```
pixi.toml
pixi.lock
pixi run start
```

Why Not Docker?

Containers are excellent for

- deployment
- cloud workflows
- CI/CD
- reproducible services
- production isolation

```
Dockerfile
apt-get
pip install
CUDA setup
X11 forwarding
volume mounts
```

But robotics research often needs

- native GUI applications
- low-friction hardware access
- ROS + ML composability
- rapid iteration across repositories
- cross-platform desktop workflows

```
pixi.toml
pixi.lock
pixi run start
```

Our goal

Native, reproducible workflows with minimal setup friction.

Limitations and Trade-offs

Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

Limitations and Trade-offs

■ Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

■ Ecosystem integration takes work

- Older scientific software may require compatibility fixes
- Packaging complex robotics stacks is still engineering effort
- Reproducibility captures complexity

Limitations and Trade-offs

■ Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

■ Ecosystem integration takes work

- Older scientific software may require compatibility fixes
- Packaging complex robotics stacks is still engineering effort
- Reproducibility captures complexity

■ Pixi is not universal

- Large organisations often have bespoke infrastructure
- Proprietary toolchains may not integrate cleanly
- Real-time and embedded workflows can require specialised environments

Limitations and Trade-offs

■ Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

■ Ecosystem integration takes work

- Older scientific software may require compatibility fixes
- Packaging complex robotics stacks is still engineering effort
- Reproducibility captures complexity

■ Pixi is not universal

- Large organisations often have bespoke infrastructure
- Proprietary toolchains may not integrate cleanly
- Real-time and embedded workflows can require specialised environments

■ But the ecosystem is improving rapidly

- Strong community support through conda-forge
- Unified CUDA support via conda-forge and NVIDIA collaboration
- Increasing convergence between robotics and ML tooling

Limitations and Trade-offs

■ Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

■ Ecosystem integration takes work

- Older scientific software may require compatibility fixes
- Packaging complex robotics stacks is still engineering effort
- Reproducibility captures complexity

■ Pixi is not universal

- Large organisations often have bespoke infrastructure
- Proprietary toolchains may not integrate cleanly
- Real-time and embedded workflows can require specialised environments

■ But the ecosystem is improving rapidly

- Strong community support through conda-forge
- Unified CUDA support via conda-forge and NVIDIA collaboration
- Increasing convergence between robotics and ML tooling

■ The goal is not perfect portability.

■ The goal is to reduce friction.

Limitations and Trade-offs

■ Cross-platform is not magic

- Linux software may still require patches on Windows or macOS
- Native libraries and build systems can behave differently across platforms
- Some robotics packages assume Ubuntu-specific tooling

■ Ecosystem integration takes work

- Older scientific software may require compatibility fixes
- Packaging complex robotics stacks is still engineering effort
- Reproducibility captures complexity

■ Pixi is not universal

- Large organisations often have bespoke infrastructure
- Proprietary toolchains may not integrate cleanly
- Real-time and embedded workflows can require specialised environments

■ But the ecosystem is improving rapidly

- Strong community support through conda-forge
- Unified CUDA support via conda-forge and NVIDIA collaboration
- Increasing convergence between robotics and ML tooling

The goal is not perfect portability.

The goal is to reduce friction.

Reproducible does not mean effortless.
It means the effort is captured.

Teasers

- Browser-native ROS via **ROS2WASM** (<https://ros2wasm.dev/>)

Teasers

- Browser-native ROS via **ROS2WASM** (<https://ros2wasm.dev/>)
- Pack environments for another machine: `pixi pack`

Teasers

- Browser-native ROS via **ROS2WASM** (<https://ros2wasm.dev/>)
- Pack environments for another machine: **pixi pack**
- **Cross-platform CI in GitHub** (<https://github.com/ruben-arts/ros-example>)

```
jobs:
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
  steps:
    - name: Setup Pixi and install environment
      uses: prefix-dev/setup-pixi@v0.9.3

    - name: Test
      run: pixi run ros2 pkg list
```

Teasers

- Browser-native ROS via **ROS2WASM** (<https://ros2wasm.dev/>)
- Pack environments for another machine: **pixi pack**
- **Cross-platform CI in GitHub** (<https://github.com/ruben-arts/ros-example>)

```
jobs:
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
  steps:
    - name: Setup Pixi and install environment
      uses: prefix-dev/setup-pixi@v0.9.3

    - name: Test
      run: pixi run ros2 pkg list
```

- conda-forge is community infrastructure: **rattler-build generate-recipe pypi some-package**

Open a PR in **conda-forge/staged-recipes**.

If a package blocks your research, packaging it can unblock the community.

Key Insights

1. Robotics is increasingly limited by software infrastructure.

Key Insights

1. Robotics is increasingly limited by software infrastructure.
2. Tasks turn research code into executable workflows.

Key Insights

1. Robotics is increasingly limited by software infrastructure.
2. Tasks turn research code into executable workflows.
3. Composable environments unify robotics and machine learning.

Key Insights

1. Robotics is increasingly limited by software infrastructure.
2. Tasks turn research code into executable workflows.
3. Composable environments unify robotics and machine learning.
4. Better packaging enables reuse, benchmarking, and collaboration.

Key Insights

1. Robotics is increasingly limited by software infrastructure.
2. Tasks turn research code into executable workflows.
3. Composable environments unify robotics and machine learning.
4. Better packaging enables reuse, benchmarking, and collaboration.
5. **Research software** is part of the contribution.

The Bigger Shift

The Bigger Shift

Dependency management is becoming core research infrastructure.

The Bigger Shift

Dependency management is becoming core research infrastructure.

Modern robotics needs:

- reproducible environments
- composable software ecosystems
- portable workflows across platforms

The Bigger Shift

Dependency management is becoming core research infrastructure.

Modern robotics needs:

- reproducible environments
- composable software ecosystems
- portable workflows across platforms

Build systems that others can run, extend, and build upon.

The Bigger Shift

Dependency management is becoming core research infrastructure.

Modern robotics needs:

- reproducible environments
- composable software ecosystems
- portable workflows across platforms

Build systems that others can run, extend, and build upon.

Questions?

Tobias.Fischer@qut.edu.au & Peter.Corke@qut.edu.au

Queensland University of Technology

Thank you!

To the **prefix.dev** team for their incredible work on Pixi, to **Silvio Traversaro** for the many hours of work on RoboStack, to **Alejandro Fontan** for his work on VSLAM-LAB, to my **co-authors**, and to the many **open-source contributors**! Also thanks to the QUT Centre for Robotics and Australian Research Council for their support.